

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 2

Roll no: 51 PRN No: 12411033 Name of Student: Manjiri Kulkarni

Subject Name : Deep Learning

Problem Statement :

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix

Algorithm :

1. Start.
2. Import the required libraries.
3. Load the Iris dataset.
4. Separate the dataset into input features (X) and target labels (y).
5. Split the dataset into training and testing sets.
6. Standardize the feature values using StandardScaler.
7. Create the MLPClassifier model with the required parameters.
8. Train the model using the training data.
9. Predict the class labels for the testing data.
10. Calculate the performance metrics (Accuracy, Precision, Recall, and F1-Score).
11. Generate the Confusion Matrix and Classification Report.
12. Plot the Loss Curve and Actual vs Predicted graph.
13. Stop.

Code :

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

from sklearn.metrics import confusion_matrix, classification_report

from sklearn.metrics import ConfusionMatrixDisplay


# Load Dataset

iris = load_iris()


data = pd.DataFrame(iris.data, columns=iris.feature_names)

data["Class"] = iris.target


print(data.head())


X = iris.data

y = iris.target


# 80% Training and 20% Testing

print("\n===== 80 : 20 Split =====")


X_train, X_test, y_train, y_test = train_test_split(

    X,
```

```
y,  
test_size=0.2,  
random_state=42  
)
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
model = MLPClassifier(  
    hidden_layer_sizes=(10,10),  
    activation="relu",  
    solver="adam",  
    max_iter=1000,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

```
prediction = model.predict(X_test)
```

```
acc1 = accuracy_score(y_test, prediction)
```

```
pre1 = precision_score(y_test, prediction, average="weighted")
```

```
rec1 = recall_score(y_test, prediction, average="weighted")
```

```
f1_1 = f1_score(y_test, prediction, average="weighted")
```

```
print("Accuracy :", acc1)
```

```
print("Precision:", pre1)
```

```
print("Recall  :", rec1)
```

```
print("F1 Score :", f1_1)
```

```
print("\nConfusion Matrix")
```

```
print(confusion_matrix(y_test, prediction))
```

```
print("\nClassification Report")
```

```
print(classification_report(y_test, prediction))
```

```
# Confusion Matrix
```

```
ConfusionMatrixDisplay.from_predictions(
```

```
    y_test,
```

```
    prediction,
```

```
    cmap="Blues"
```

```
)
```

```
plt.title("80-20 Confusion Matrix")
```

```
plt.show()
```

```
# Loss Curve
```

```
plt.figure(figsize=(6,4))  
plt.plot(model.loss_curve_, marker="o")  
plt.title("80-20 Loss Curve")  
plt.xlabel("Iterations")  
plt.ylabel("Loss")  
plt.grid(True)  
plt.show()
```

```
# Actual vs Predicted
```

```
plt.figure(figsize=(8,4))  
  
plt.plot(y_test, marker='o', label='Actual')  
plt.plot(prediction, marker='x', label='Predicted')  
  
plt.xlabel("Sample Number")  
plt.ylabel("Class")  
plt.title("Actual vs Predicted")  
plt.yticks([0, 1, 2], iris.target_names)  
plt.legend()  
plt.grid(True)
```

```
plt.show()
```

```
# 90% Training and 10% Testing
```

```
print("\n=== 90 : 10 Split ===")
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.1,
```

```
    random_state=42
```

```
)
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
model = MLPClassifier(
```

```
    hidden_layer_sizes=(10,10),
```

```
    activation="relu",
```

```
    solver="adam",
```

```
    max_iter=1000,
```

```
    random_state=42
```

```
)
```

```
model.fit(X_train, y_train)
```

```
prediction = model.predict(X_test)
```

```
acc2 = accuracy_score(y_test, prediction)
```

```
pre2 = precision_score(y_test, prediction, average="weighted")
```

```
rec2 = recall_score(y_test, prediction, average="weighted")
```

```
f1_2 = f1_score(y_test, prediction, average="weighted")
```

```
print("Accuracy :", acc2)
```

```
print("Precision:", pre2)
```

```
print("Recall  :", rec2)
```

```
print("F1 Score :", f1_2)
```

```
print("\nConfusion Matrix")
```

```
print(confusion_matrix(y_test, prediction))
```

```
print("\nClassification Report")
```

```
print(classification_report(y_test, prediction))
```

```
# Confusion Matrix
```

```
ConfusionMatrixDisplay.from_predictions(
```

```
y_test,  
prediction,  
cmap="Blues"  
)
```

```
plt.title("90-10 Confusion Matrix")  
plt.show()
```

```
# Loss Curve
```

```
plt.figure(figsize=(6,4))  
plt.plot(model.loss_curve_, marker="o")  
plt.title("90-10 Loss Curve")  
plt.xlabel("Iterations")  
plt.ylabel("Loss")  
plt.grid(True)  
plt.show()
```

```
# Actual vs Predicted
```

```
plt.figure(figsize=(8,4))  
  
plt.plot(y_test, marker='o', label='Actual')  
plt.plot(prediction, marker='x', label='Predicted')
```

```
plt.xlabel("Sample Number")
plt.ylabel("Class")
plt.title("Actual vs Predicted")
plt.yticks([0, 1, 2], iris.target_names)
plt.legend()
plt.grid(True)

plt.show()
```

```
# Comparison Table
```

```
result = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1 Score"],
    "80-20": [acc1, pre1, rec1, f1_1],
    "90-10": [acc2, pre2, rec2, f1_2]
})
```

```
result["Difference"] = result["80-20"] - result["90-10"]
```

```
print("\nComparison of Both Splits")
```

```
print(result)
```

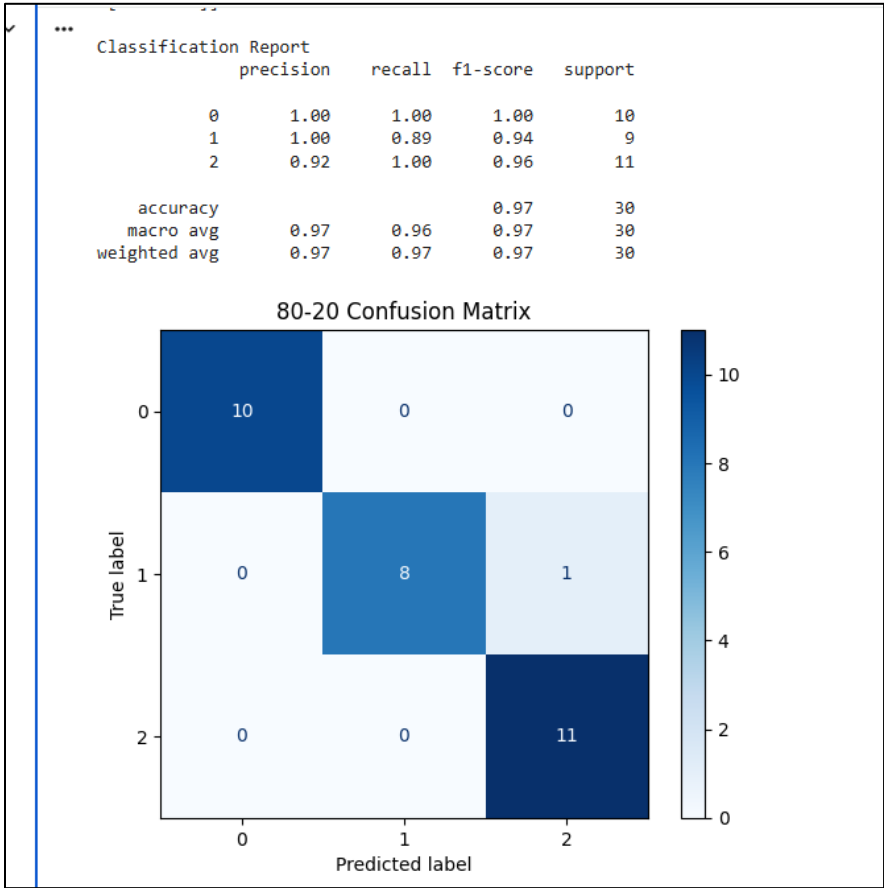
Output :

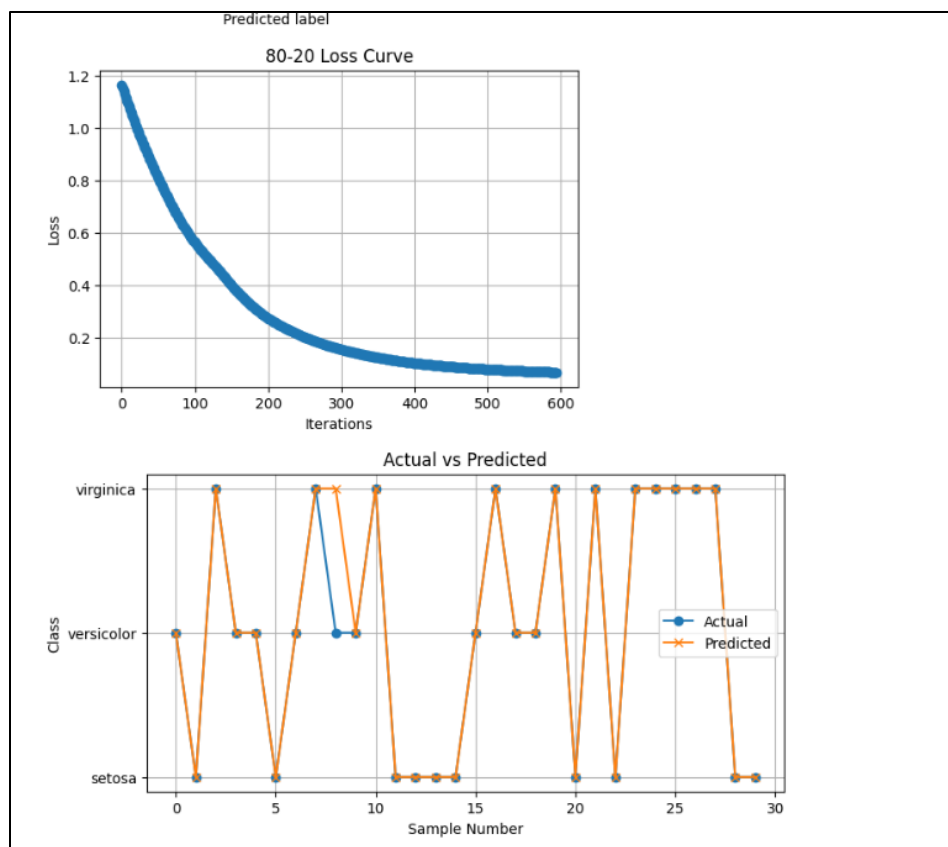
```
...      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0          5.1             3.5             1.4             0.2
1          4.9             3.0             1.4             0.2
2          4.7             3.2             1.3             0.2
3          4.6             3.1             1.5             0.2
4          5.0             3.6             1.4             0.2

      Class
0         0
1         0
2         0
3         0
4         0

===== 80 : 20 Split =====
Accuracy : 0.9666666666666667
Precision: 0.9694444444444444
Recall   : 0.9666666666666667
F1 Score : 0.9664109121909632

Confusion Matrix
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
```





===== 90 : 10 Split =====

Accuracy : 1.0

Precision: 1.0

Recall : 1.0

F1 Score : 1.0

Confusion Matrix

[[6 0 0]

[0 6 0]

[0 0 3]]

Classification Report

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6
1	1.00	1.00	1.00	6
2	1.00	1.00	1.00	3
accuracy			1.00	15
macro avg	1.00	1.00	1.00	15
weighted avg	1.00	1.00	1.00	15

